

# Improving the performance of parallel programs

# Bibliography

- Culler et al – Chapter 3
- Grama et al – Chapter 3

# Parallelization process

1. Decomposition
2. Assignment
3. Orchestration
4. Mapping

# Performance of parallel programs

- Optimize the performance = refine choices for decomposition, assignment, orchestration and mapping
- Goals:
  - Balance workload
  - Reduce communication
  - Reduce overhead introduced to increase parallelism, manage assignment, reduce communication, etc.
- **Optimization goals are at odds with each other!**
- Techniques to achieve these goals

# Load balancing

- Ensuring that every processor does the same amount of work
- Goals for processes:
  - Same amount of work
  - Execute at the same time
  - Minimize the waiting time spent for sync
  - Minimize the serialization of processes due to mutual exclusion or dependencies

# Load balancing issues

- Identifying enough concurrency in decomposition, and overcoming Amdahl's Law.
- Deciding how to manage the concurrency (statically or dynamically).
- Determining the granularity at which to exploit the concurrency.
- Reducing serialization and synchronization cost.

# How to identify concurrency?

# Decomposition techniques

- Identify enough concurrency
- Parallelizing loops leads to data parallelism (same calculation on elements of a large data structure)
- Function (task) parallelism – different calculations performed in parallel on different or on the same data
- Too much? -> restrict the available concurrency by determining the granularity of tasks



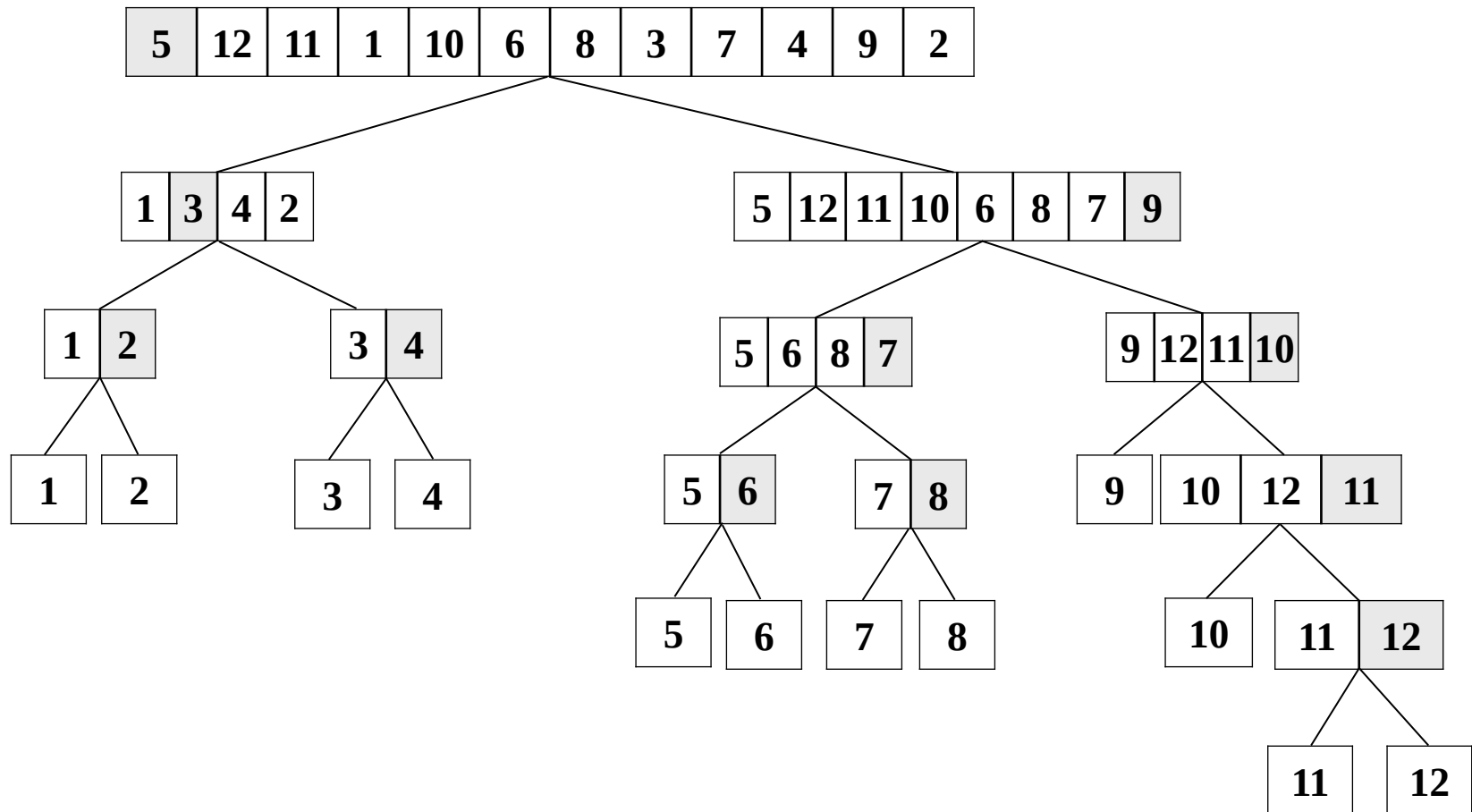
# Decomposition techniques

- Allow to identify the concurrency in the problem
- Split the computation into a set of tasks
- Techniques
  - recursive decomposition
  - data decomposition
  - exploratory decomposition
  - speculative decomposition

# Recursive decomposition

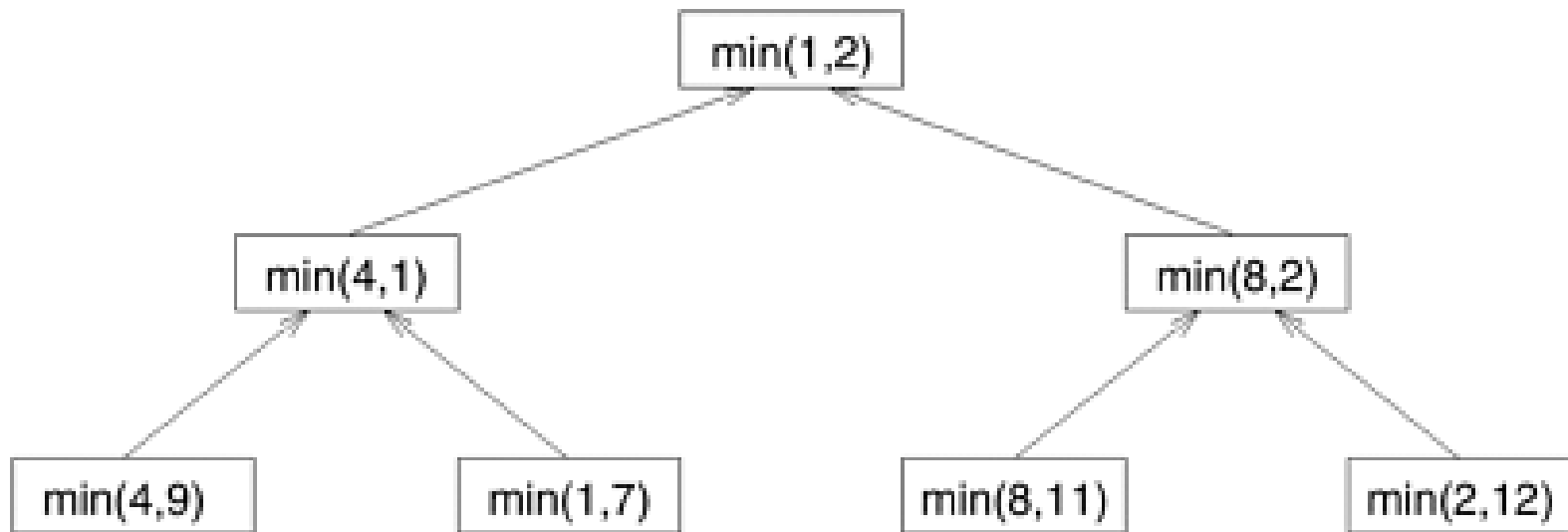
- Suitable for problems that can be solved using divide-and-conquer strategy
  - divide-and-conquer = problem divided into a set of independent subproblems, each subproblem is divided again ... at the end the partial results are combined
- Examples
  - quicksort – the sequential algorithm uses divide-and-conquer
  - minimum – the sequential algorithm is modified to use divide-and-conquer

# Quicksort



# Minimum

- Restructure computation as a divide-and-conquer algorithm



# Data decomposition

- Suitable for finding concurrency in algorithms that operate on large data structures
  - decomposition of computations is done into two steps
    - step 1: data is partitioned
    - step 2: data partitioning is used to induce a partitioning of the computations into tasks (operations performed by the tasks on the data partitions are similar or chosen from a small set of operations)
  - data partitioning can be done in various ways
    - partitioning output data
    - partitioning input data
    - partitioning both output and input data
    - partitioning intermediate data

# Partitioning output data

- Partitioning of output data directly induces a decomposition of the problem into tasks
- Each task has to produce a part of the output
- Each partition of the output can be computed independently of others as a function of the input
- Example: matrix multiplication
  - partition output matrix C into four submatrices
  - each task computes one submatrix

$$\begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix} \cdot \begin{pmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{pmatrix} \rightarrow \begin{pmatrix} C_{1,1} & C_{1,2} \\ C_{2,1} & C_{2,2} \end{pmatrix}$$

$$\text{Task 1: } C_{1,1} = A_{1,1} B_{1,1} + A_{1,2} B_{2,1}$$

$$\text{Task 2: } C_{1,2} = A_{1,1} B_{1,2} + A_{1,2} B_{2,2}$$

$$\text{Task 3: } C_{2,1} = A_{2,1} B_{1,1} + A_{2,2} B_{2,1}$$

$$\text{Task 4: } C_{2,2} = A_{2,1} B_{1,2} + A_{2,2} B_{2,2}$$

# Partitioning output data

- a given data decomposition does not result into a unique task decomposition
- example – two task decomposition for the same data decomposition

| Decomposition I                               | Decomposition II                              |
|-----------------------------------------------|-----------------------------------------------|
| Task 1: $C_{1,1} = A_{1,1} B_{1,1}$           | Task 1: $C_{1,1} = A_{1,1} B_{1,1}$           |
| Task 2: $C_{1,1} = C_{1,1} + A_{1,2} B_{2,1}$ | Task 2: $C_{1,1} = C_{1,1} + A_{1,2} B_{2,1}$ |
| Task 3: $C_{1,2} = A_{1,1} B_{1,2}$           | Task 3: $C_{1,2} = A_{1,2} B_{2,2}$           |
| Task 4: $C_{1,2} = C_{1,2} + A_{1,2} B_{2,2}$ | Task 4: $C_{1,2} = C_{1,2} + A_{1,1} B_{1,2}$ |
| Task 5: $C_{2,1} = A_{2,1} B_{1,1}$           | Task 5: $C_{2,1} = A_{2,2} B_{2,1}$           |
| Task 6: $C_{2,1} = C_{2,1} + A_{2,2} B_{2,1}$ | Task 6: $C_{2,1} = C_{2,1} + A_{2,1} B_{1,1}$ |
| Task 7: $C_{2,2} = A_{2,1} B_{1,2}$           | Task 7: $C_{2,2} = A_{2,1} B_{1,2}$           |
| Task 8: $C_{2,2} = C_{2,2} + A_{2,2} B_{2,2}$ | Task 8: $C_{2,2} = C_{2,2} + A_{2,2} B_{2,2}$ |

# Partitioning input data

- Partitioning of input data directly induces a decomposition of the problem into tasks
  - a task is created for each partition of input data
  - combine the results
- Example: computing the sum of  $N$  elements using  $p$  processors
  - partition the input into  $p$  subsets of (nearly) equal size
  - each task computes the sum for one subset
  - $p$  partial sums are added to produce the final result



# Partitioning intermediate data

- Suitable for algorithms organized in multiple steps (the output of one step is the input for next step)
- Sometimes leads to higher concurrency than partitioning input or output data
- Sometimes modifications of serial algorithms are needed as intermediate data are not explicitly generated in the serial algorithm

# Partitioning intermediate data

## Stage I

$$\begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix} \cdot \begin{pmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{pmatrix} \rightarrow \begin{pmatrix} \begin{pmatrix} D_{1,1,1} & D_{1,1,2} \\ D_{1,2,2} & D_{1,2,2} \end{pmatrix} \\ \begin{pmatrix} D_{2,1,1} & D_{2,1,2} \\ D_{2,2,2} & D_{2,2,2} \end{pmatrix} \end{pmatrix}$$

$$\begin{pmatrix} D_{1,1,1} & D_{1,1,2} \\ D_{1,2,2} & D_{1,2,2} \end{pmatrix} + \begin{pmatrix} D_{2,1,1} & D_{2,1,2} \\ D_{2,2,1} & D_{2,2,2} \end{pmatrix} \rightarrow \begin{pmatrix} C_{1,1} & C_{1,2} \\ C_{2,1} & C_{2,2} \end{pmatrix}$$

## Stage II

A decomposition induced by a partitioning of  $D$

$$\text{Task 01: } D_{1,1,1} = A_{1,1}B_{1,1}$$

$$\text{Task 02: } D_{2,1,1} = A_{1,2}B_{2,1}$$

$$\text{Task 03: } D_{1,1,2} = A_{1,1}B_{1,2}$$

$$\text{Task 04: } D_{2,1,2} = A_{1,2}B_{2,2}$$

$$\text{Task 05: } D_{1,2,1} = A_{2,1}B_{1,1}$$

$$\text{Task 06: } D_{2,2,1} = A_{2,2}B_{2,1}$$

$$\text{Task 07: } D_{1,2,2} = A_{2,1}B_{1,2}$$

$$\text{Task 08: } D_{2,2,2} = A_{2,2}B_{2,2}$$

$$\text{Task 09: } C_{1,1} = D_{1,1,1} + D_{2,1,1}$$

$$\text{Task 10: } C_{1,2} = D_{1,1,2} + D_{2,1,2}$$

$$\text{Task 11: } C_{2,1} = D_{1,2,1} + D_{2,2,1}$$

$$\text{Task 12: } C_{2,2} = D_{1,2,2} + D_{2,2,2}$$

# Exploratory decomposition

- suitable for problems which search a space for the solutions
  - partition the search space into smaller parts
  - concurrently search each part
  - example: 15-puzzle problem

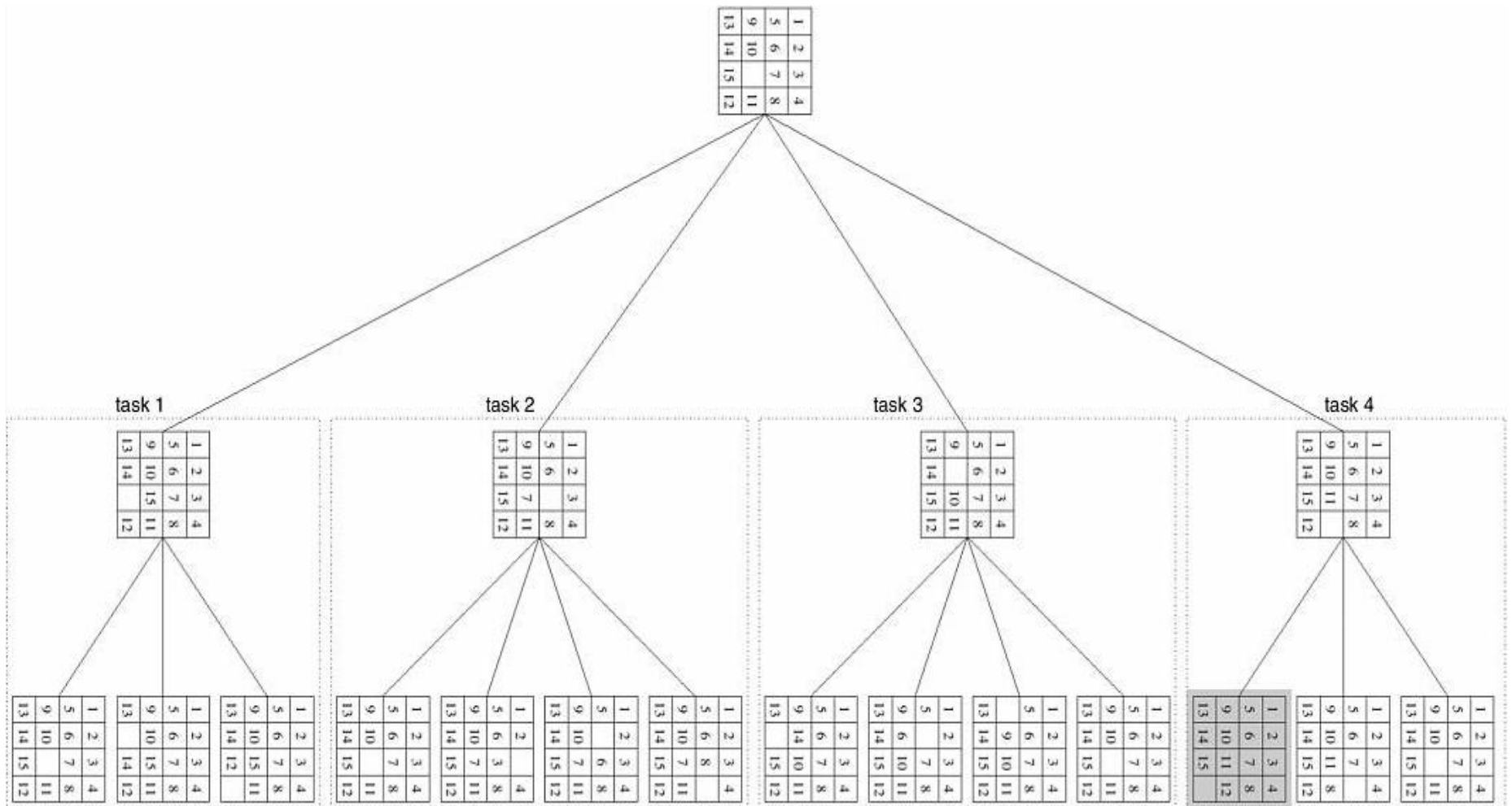
|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

|    |    |    |    |
|----|----|----|----|
| 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  |
| 9  | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

# 15 puzzle



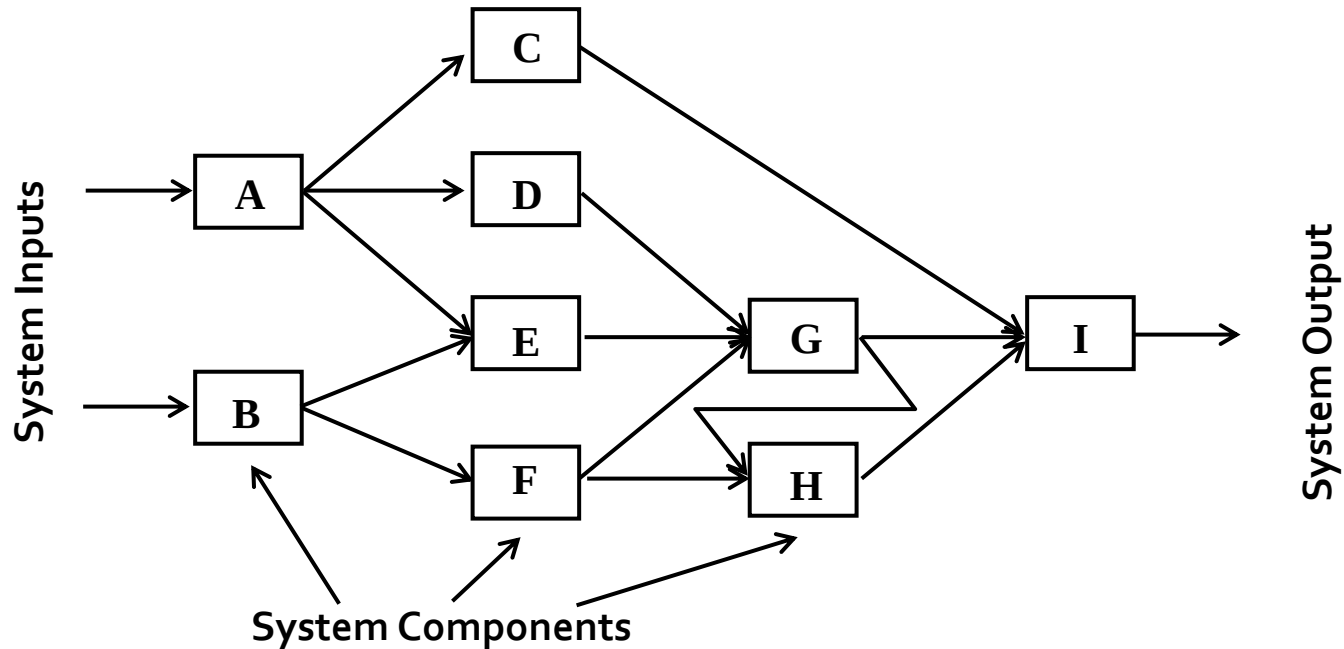
# Exploratory decomposition

- Solution for 15-puzzle is based on tree-search
- Solving the problem in parallel
  - few levels of configurations are generated sequentially
  - each node is assigned to a task to expand it further → concurrent tasks
  - the task which finds the solution notifies the other tasks
- Is exploratory decomposition identical to data decomposition (search space = data)?
  - data decomposition: tasks induced by data decomposition are fully performed and each task perform useful computation towards the solution of the problem
  - exploratory decomposition: unfinished tasks can be terminated when the overall solution is found

# Speculative decomposition

- Suitable for the situation in which a program may take one of many branches ( $B_1, \dots, B_n$ ) depending on the output of another computation (C) that precedes it
  - $C, B_1, \dots, B_n$  go in parallel
  - similar with evaluating branches of a switch statement in parallel before the input for switch is available
  - computations of some branches are discarded
  - minimize of wasted computation – consider the most likely / promising branch(es)

# Simulator – event based



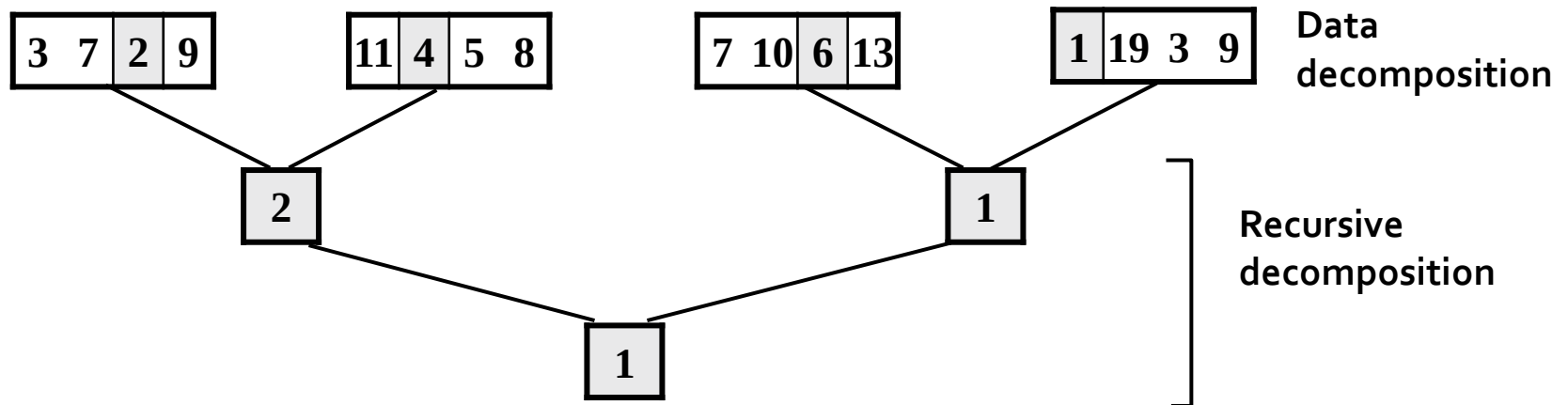
# Speculative decomposition

- Is speculative decomposition identical to exploratory decomposition?
  - exploratory decomposition performs smaller, equal or greater amount of work compared to the serial algorithm (depending on the location of solution in the search space)
  - speculative decomposition performs equal or greater amount the work compared to the serial algorithm



# Hybrid decomposition

- combine decomposition techniques
- computation is structured in multiple stages, apply different types of decomposition for different stage



# Task characteristics

- Task generation
  - static: data partitioning, recursive decomposition (ex: minimum), exploratory decomp.
  - dynamic: recursive decomposition (ex: quicksort), exploratory decomp.
- Task size/granularity (time required to complete the task)
  - uniform
  - non-uniform
- Knowledge of task size
  - known
  - not known
- Size of data
  - input data
  - output data
  - amount of computation

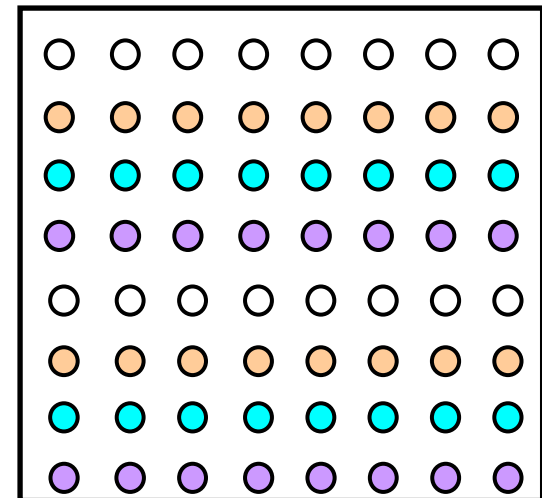
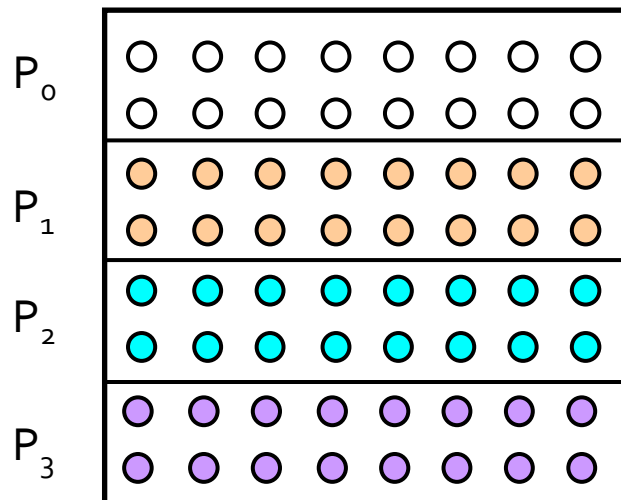
# Inter-task interactions characteristics

- static vs dynamic
  - static: interactions are predetermined
  - dynamic: interactions are not known prior to the execution
- regular vs irregular
  - regular: structure can be exploited for efficient implementation
  - irregular: structure is unknown
- read-only vs read-write
- one-way vs two-way
  - one-way: one task initiates the communication and completes it without interrupting the other task
  - two-way: data needed by a task is explicitly supplied by another task

# How to manage concurrency?

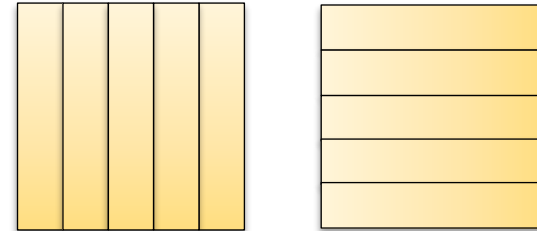
# Static assignment

- Assignment of work to processes is pre-determined
- Static task generation => static assignment
- Solver example:

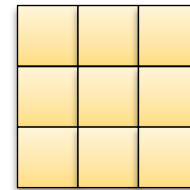


# Block distributions

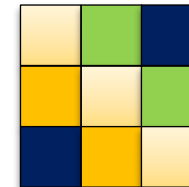
- 1D-block



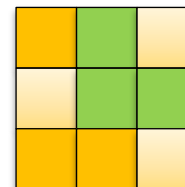
- 2D-block



- Block-cyclic

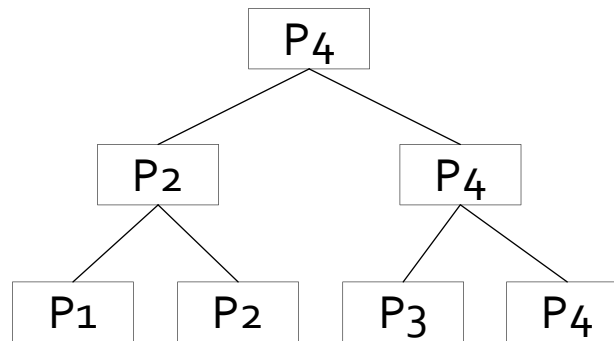


- Randomized block



# Task partitioning

- Task dependency graph
- Optimal assignment – NP-complete problem
- Find approximate solution



# Static assignment

- Advantages: simple, zero runtime overhead
- Requirements:
  - The cost and amount of work is predictable
    - Processes have the same (known) execution time
    - Processes have different (known) execution time
  - When statistics about execution time are known
  - No interference from other applications



# Semi-static assignment

- Cost of work is predictable for near-term future
- Application periodically profiles itself and re-computes assignment
- Good for environments that evolve slowly

# Dynamic assignment

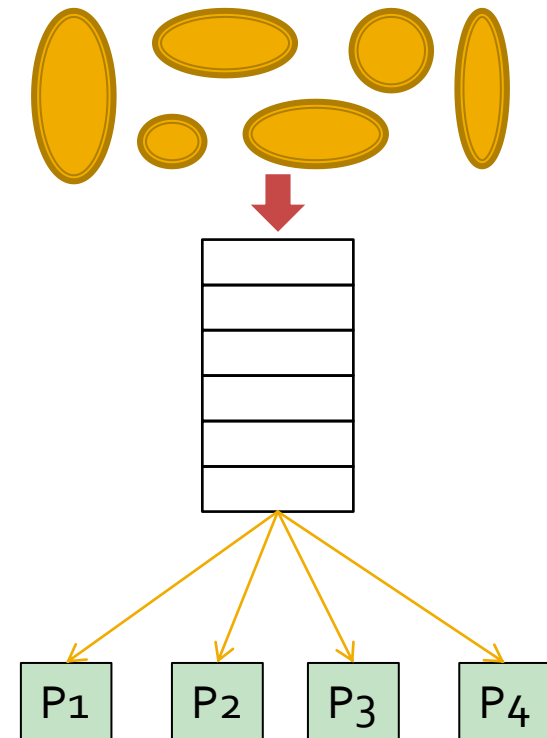
- Work distribution or the system environment is too unpredictable to periodically re-compute the assignment
- Program determines the assignment dynamically at runtime to ensure a well distributed load

# Dynamic assignment

- Self-scheduling (parallel loop):
  - Shared loop counter
  - Processes increment the counter (atomically), execute the iteration, and repeat...
- Collection of work queues
  - Tasks are inserted into queues
  - Processes remove tasks and execute them

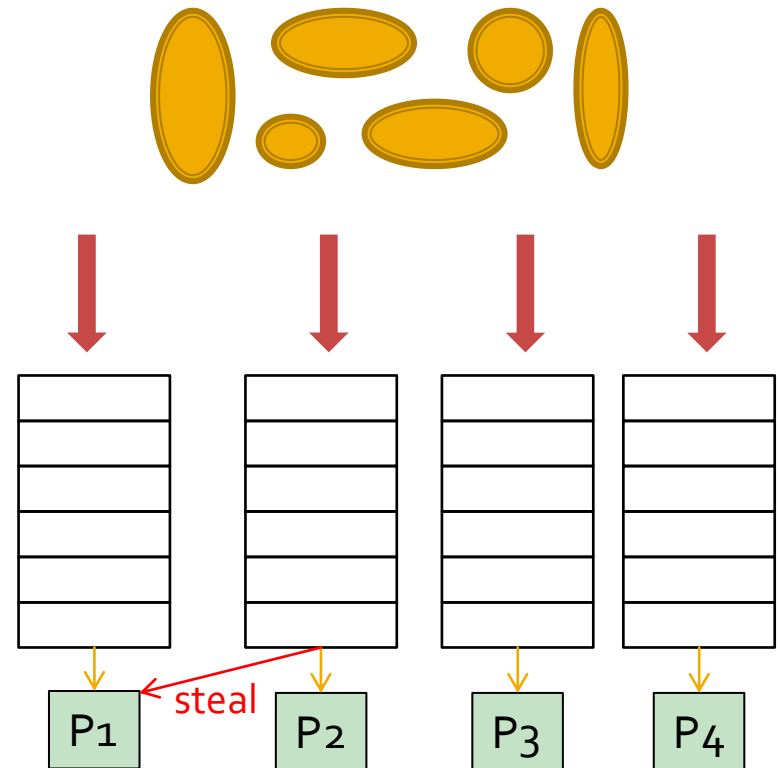
# Dynamic assignment – shared queue

- Single centralized queue
- Simple
- All processes access the same queue:
  - Contention
  - Mutual exclusion
- Performance bottleneck:
  - Many small tasks
  - Large numbers of processes
- Load imbalance: large tasks



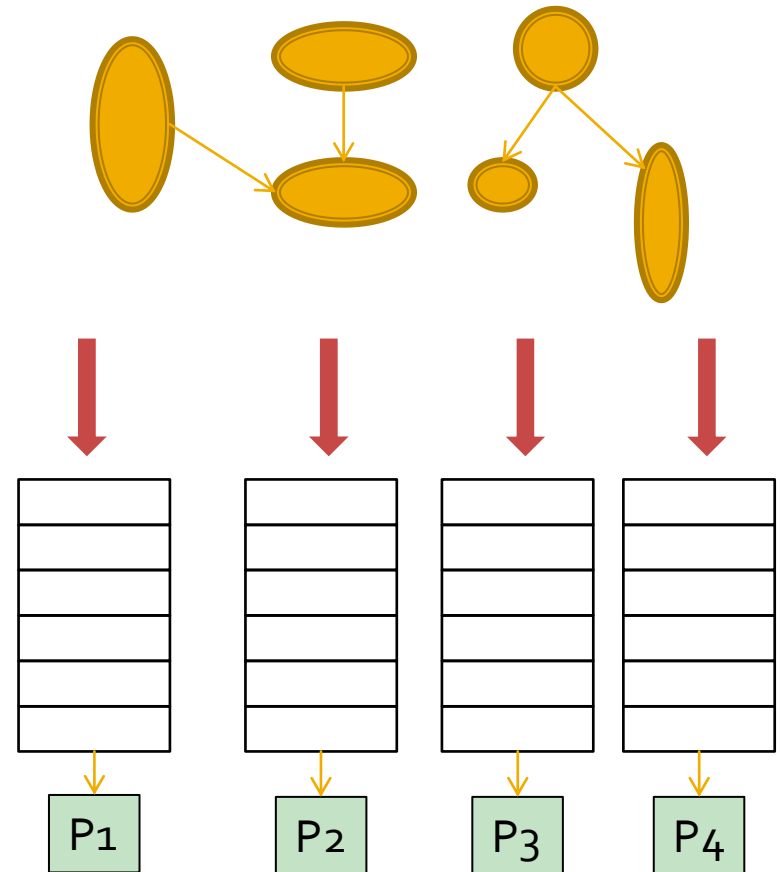
# Dynamic assignment – distributed queue

- Avoids synchronization
- Expensive management
- Intelligent assignment of tasks to queues: can minimize communication
- Task stealing: generates contention
  - Minimize stealing
  - How much to steal
  - Where to steal from
  - Termination detection



# How about task dependencies?

- Task is removed from queue and assigned to process after all dependencies are satisfied



# Task granularity

# Task granularity

- Load imbalance caused by
  - Task dependences
  - Granularity of the largest task
- Granularity = amount of work associated with a task, measured by the number of instructions (execution time)
- Small tasks:
  - Better load balancing
  - Larger management overhead
  - More contention
  - More communication



# Task granularity

- Dynamic task queuing
  - Large management overhead with small tasks
  - Contention and communication increase with smaller tasks
- Static assignment
  - Smaller effect on task management overhead
  - Larger tasks can cause load imbalance

# Questions

- Define the term “Partitioning” in the context of parallel programs design.
- Which are the issues of task stealing (distributed task queues)?